ML Laboratory 02: Logistic Regression

From Probabilities to Class Decisions

Table of contents

1 Objective 1

2 Before starting 2

3 Essential theory 2
- 3.1 Model and parameters . . . 2
- 3.2 Turn probability into class decision . . . 3
- 3.3 Training cost . . . 3
- 3.4 Test evaluation . . . 4

4 Practical work 4
- 4.1 Task 1: see the model in two dimensions . . . 4
- 4.2 Task 2: inspect the Titanic data and baseline . . . 5
- 4.3 Task 3: fit a one-predictor model . . . 5
- 4.4 Task 4: add age and fare . . . 6
- 4.5 Task 5: calculate a prediction yourself . . . 6
- 4.6 Task 6: apply the workflow to a new dataset . . . 7

5 Final questions 7

1 Objective

Fit a binary classifier, turn its probabilities into class decisions, and check its predictions on testing data.

2 Before starting

Use **MATLAB R2019a or newer with Statistics and Machine Learning Toolbox**. Keep the `data` folder beside `lab2_starter.m`.

Open the provided starter file and run its `%%` sections in order, using **Run Section** command. Complete **TODO A–D**, then write your own code in sections **5–6**. An unfinished task deliberately stops the script; if you clear the workspace, rerun the earlier sections.

Collect the deliverables and screenshots in one short document: `Lab2_Name_Group.pdf`. Include your name and group. Use readable screenshots with short captions, not a theory write-up.

3 Essential theory

3.1 Model and parameters

In Lab 1 (Linear Regression) the output was a real number (e.g. 3.56, -4.3).

Here the target is to predict a **class**, encoded as $y = 0$ (class 0) or $y = 1$ (class 1).

Logistic regression operates in two steps:

1. first compute a linear score z, using a weighted sum of the inputs and a bias term (just like linear regression)
2. then apply the sigmoid function $\sigma(z)$ to that score

$$z = b + w_1 x_1 + \cdots + w_n x_n, \qquad p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The model output is a p which estimates the probability that x belongs to class 1.

The parameters of Logistic Regression are the **bias** b and the **weights** $w_1, \ldots, w_n$, which are the same as in linear regression.

3.2 Turn probability into class decision

The model returns a value $p \in [0, 1]$, which we interpret as the probability that the input belongs to class 1.

To turn the value into a class decision, we compare p with a threshold. With threshold $T = 0.5$, our decision rule is

$$\hat{y} = \begin{cases} 1, & p \geq 0.5, \\ 0, & p < 0.5. \end{cases}$$

For example, $p = 0.7$ gives class 1, and $p = 0.45$ gives class 0. The threshold can be changed to favor one class over the other.

The boundary $p = 0.5$ is where $z = 0$. This happens when $z = b + w_1 x_1 + \cdots + w_n x_n = 0$. This defined a **linear decision boundary** in the input space:

- For two inputs, this frontier is a line;
- For three inputs, it is a plane
- For more inputs, it is a hyperplane.

Such a linear decision boundary is very simple, but it is often sufficient.

3.3 Training cost

The training process optimizes the parameters in order to minimize the cost function.

For logistic regression, the cost function is the **mean binary cross-entropy**:

$$J_{\text{train}} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where y_i is the true class of input vector i (0 or 1), and p_i is the predicted probability for it.

The cost function can be understood as follows:

- when the true class is $y_i = 1$, the ideal output should be $p = 1$, and we penalize a value closer to 0:
$$\text{Cost } J(i) = -\log(p_i) = -\log(p_i)$$
- when the true class is $y_i = 0$, the ideal output should be $p = 0$, and we penalize a value closer to 1:
$$\text{Cost } J(i) = -\log(1 - p_i) = -\log(1 - p_i)$$

We can use MATLAB's `fitglm()` function for training.

```
mdl = fitglm(Xtrain, ytrain, 'Distribution', 'binomial');
coeff = mdl.Coefficients.Estimate; % Bias first, then predictor weights
pTest = predict(mdl, Xtest);      % Probabilities, not class labels
```

`Xtrain` and `Xtest` are training and test input data, arranged as rows (rows are observations; columns are predictors). `ytrain` is a column vector of training targets. For example, with three predictors, `Xtrain` is `N`-by-`3`, `ytrain` is `N`-by-`1`, and the model has four parameters (bias and three weights).

3.4 Test evaluation

For a fair evaluation, we must **hold out** some rows of the data for testing. Test data is not used for training.

We evaluate the model's performance using the **accuracy** of class decisions for the test data:

$$\text{accuracy} = \frac{\text{number of correct class decisions}}{\text{number of test observations}}.$$

The outputs of a classifier can be summarized in a **confusion matrix**, in which we count the number of correct and incorrect predictions for each class, with **rows = actual class**, **columns = predicted class**:

	Predicted 0	Predicted 1
Actual 0	True negatives	False positives
Actual 1	False negatives	True positives

It is important to use a balanced test set, if possible, or to inspect the confusion matrix rather than just the accuracy. When there are far more examples of one class than the other, accuracy alone can hide poor predictions.

4 Practical work

4.1 Task 1: see the model in two dimensions

Run sections 0–1. The supplied synthetic data contain two overlapping classes. The starter fits a model and plots its probabilities and decision boundary. This is an illustration using all 120 points, not a held-out performance test.

1. Identify the bias and two weights in the printed output.
2. Locate the boundary on the plot. What probability does it represent?
3. Read the probability and class for the example point. Why are they different kinds of output? Why need the boundary not classify every point correctly?

Deliverables: Figure 1 and the printed coefficients, example probability, and class.

4.2 Task 2: inspect the Titanic data and baseline

Let's evaluate a simple baseline "dumb" model which ignores all passenger features and always predicts the most common class in the training data. Later, we will compare the trained model with this baseline to see how much it improves on this simple rule.

Run section 2. The prepared data describe historical Titanic passengers. The target is `survived`: **0 = did not survive**, **1 = survived**.

CSV column	Meaning	Use
1: `sex_code`	Sex (binary encoded)	Predictor
2: `age_scaled`	Age (transformed)	Predictor
3: `fare_scaled`	Fare (transformed)	Predictor
4: `survived`	Observed outcome, 0 or 1	Target
5: `is_train`	1 = training; 0 = test	Split metadata only
6: `passenger_id`	Passenger identifier	Identifier only

The dataset is publicly available from https://www.kaggle.com/datasets/azeembootwala/titanic and is already preprocessed. No preprocessing is required during this lab.

1. Inspect the first five rows and the printed class counts. Check that `Xtrain` is `791 3` and `Xtest` is `100 3`.
2. Identify the baseline's predicted class and test accuracy. Why is its class chosen from the training targets, rather than the test targets?
3. Explain why columns 5 and 6 must not be supplied as predictors.

Deliverables: the displayed matrix sizes, class counts, baseline class, and baseline test accuracy.

4.3 Task 3: fit a one-predictor model

Complete TODO A–C in section 3, then run it. This model uses only `sex_code`.

1. **TODO A:** select column 1 of `Xtrain` into `xSingle`.

2. Read the supplied fitting and prediction calls. Which rows fit the parameters? Which rows receive test predictions?
3. **TODO B:** convert `pSingle` to class labels using threshold 0.5.
4. **TODO C:** compute the fraction of correct predictions. In MATLAB, `mean(a == b)` gives the fraction of matching elements for column vectors.
5. Read the bias, weight, and test accuracy. Compare accuracy with the baseline. The printed accuracy is a percentage; your expression should return a fraction between 0 and 1.

Deliverables: completed TODO A–C, the coefficients, and test accuracy.

4.4 Task 4: add age and fare

Complete TODO D in section 4, then run it.

1. **TODO D:** fit logistic regression using all three columns of `Xtrain` and the same `ytrain`. Use the fitting call in section 3 as a pattern.
2. Inspect the comparison of training costs and test accuracies. Which quantity was minimized during fitting? Which quantity judges held-out class decisions?
3. Inspect Figure 2. How many survivors were predicted not to survive? How many non-survivors were predicted to survive? Check that all four counts sum to 100.
4. Compare both prescribed models with the baseline. Do the extra predictors improve test accuracy **on this split**? A tie or worse result is valid.

Keep the supplied split and threshold. Do not repeatedly try additional predictors to improve this test score: that would turn the test set into tuning data. The historical data and undocumented upstream preparation limit what this small experiment can establish about future predictions.

Deliverables: completed TODO D, the printed comparison, and Figure 2.

4.5 Task 5: calculate a prediction yourself

Write three lines in starter section 5 to manually compute a prediction. The supplied `xManual` is one test passenger's three predictors; `bManual` and `wManual` are the fitted bias and weight vector from Task 4.

1. Compute the linear score `zManual` using the bias and weights.
2. Apply the sigmoid to obtain `pManual`.
3. Use threshold 0.5 to obtain `classManual`.

Use your own expressions, not `predict`, for these three lines. The supplied check compares your result with `predict`. Reminder: use MATLAB's `exp(z)` to compute e^z; a `1`-by-`3` row times a `3`-by-`1` column gives one number.

Deliverables: your three lines and the printed manual/library predictions.

4.6 Task 6: apply the workflow to a new dataset

Write about six lines of code to apply logistic regression to a new dataset, the Lower Back Pain Symptoms dataset.

Read the description on the source page https://www.kaggle.com/datasets/sammy123/lower-back-pain-symptoms-dataset

The dataset is available in the `data` folder as `spine_data.csv`, already preprocessed. It has a binary target label, indicating whether a patient has normal or abnormal spine conditions (**Normal = 0, Abnormal = 1**).

The code to load the CSV file and split into train and test sets is already provided: `XspineTrain` is `248`-by-`12`, `XspineTest` is `62`-by-`12`; targets are `yspineTrain` and `yspineTest`. Use the earlier Titanic code as a reference, but write the following yourself:

1. Fit a new logistic model, `mdlSpine`, using only the Spine training data.
2. Predict test probabilities into `pSpine`.
3. Convert them into `classSpine` at threshold 0.5.
4. Compute the test accuracy fraction, `accuracySpine`.
5. Count false negatives in `falseNegSpine`: actual 1, predicted 0.
6. Count false positives in `falsePosSpine`: actual 0, predicted 1.

Use `sum(condition)` to count matches and `&` to combine elementwise conditions.

Some predictors are nearly redundant; MATLAB may warn about conditioning or rank deficiency.

Deliverables: screenshot with your code, and the printed outputs (accuracy and error counts).

5 Final questions

1. Suppose a wrong medical decision has a cost, and that the cost of classiyfing an abnormal patient as normal is different from the cost of classifying a normal patient as abnormal. Would threshold 0.5 necessarily be the best choice in this case, or should a different threshold be used?